

In this mini project we were tasked with designing a multi-cycle RISC-V processor. The design must implement all of the instructions in the base RV32I instruction set except for `ecall`, `ebreak`, `csrrw`, `csrrs`, `csrrc`, `csrrwi`, `csrrsi`, and `csrrci`. A SystemVerilog test bench and simulation results are provided below. Additionally, Icarus Verilog (iverilog) verifies proper operation of our processor with representative examples of each class of RV32I instructions.

Repo Link: https://github.com/MateoBrancoliW/CompArcMP4_FA2025

Processor Design

Our processor's design is a RISC-V adaptation of the Patterson & Hennessy multicycle processor architecture which for the most part maintains structure. There is a similar adaptation done by Harris & Harris and we relied on theirs as well due to its clearer control unit. We restructured the immediate block for RISC-V and some more logic. RISC-V immediates come in 5 formats:

- I-type: 12-bit signed
- S-type: split imm[11:5] and imm[4:0]
- B-type: branch immediate, split and shifted
- U-type: 20-bit upper immediate
- J-type: 20-bit jump immediate, shuffled

Abstracted Process flow of Multi-Cycle RISC-V Processor

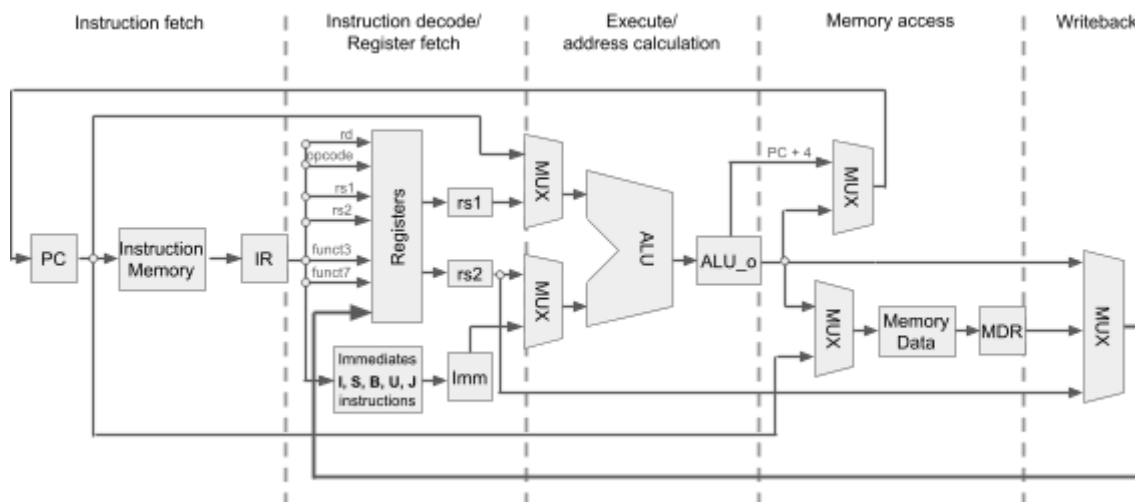


Figure 1. illustrates the abstracted datapath of a multi-cycle RISC-V processor, showing how a single instruction flows through the machine over several cycles. The design partitions execution into five conceptual phases—Instruction Fetch, Instruction Decode/Register Fetch, Execute/Address Calculation, Memory Access, and Writeback—each reusing shared hardware components. Temporary registers such as the Instruction Register (IR), ALU output register (ALU_o), and Memory Data Register (MDR) preserve intermediate results across cycles.

Control Unit

All execution of instructions are coordinated and directed in the control unit, which consists of an Instruction Decoder, a main FSM, and an ALU Decoder. The Instruction Decoder defines `imm_src`, the opcode's immediate format class, for use in `immediate_extender` to construct the actual sign-extended 32-bit immediate. Our main FSM implements microstates for each step of instruction execution. Each microstate corresponds to a certain clock cycle, and the next microstate is already set. Datapath registers are updated on the next rising clock according to enabled signals. The ALU Decoder produces `alu_control`, the specific ALU operation that the datapath will execute. `Alu_op` indicates the instruction group: `2'b00` for unconditional add used in address calculations; `2'b01` for unconditional subs used in branch comparisons; and `2'b10` for R-/I- type operations where both `funct3` and `funct7` must be examined. Unique case is utilized to ensure parallel decoding and to guarantee that all cases are mutually exclusive. Below is the diagram that shows the full multicycle processor from the harris & harris textbook:

Complete Multi-Cycle RISC-V Processor

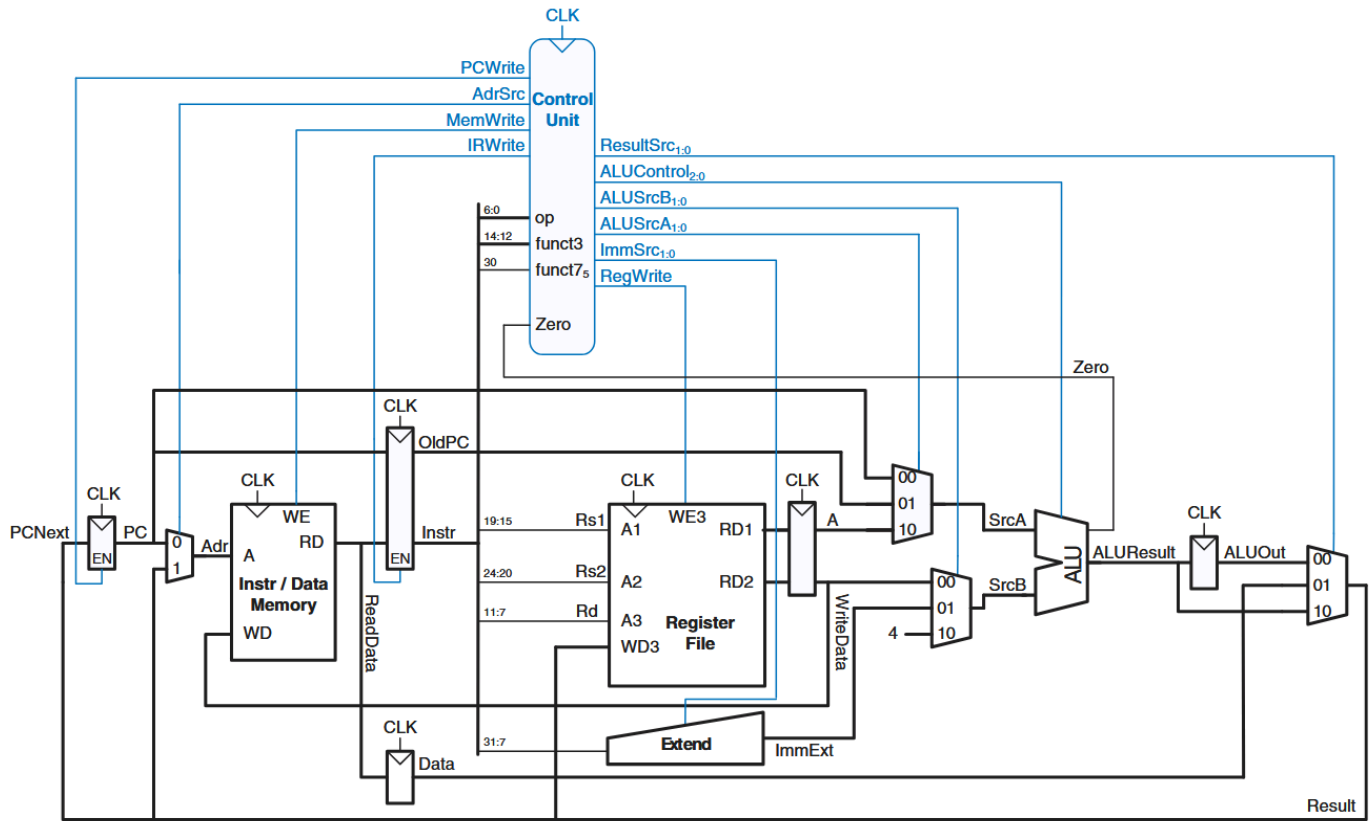
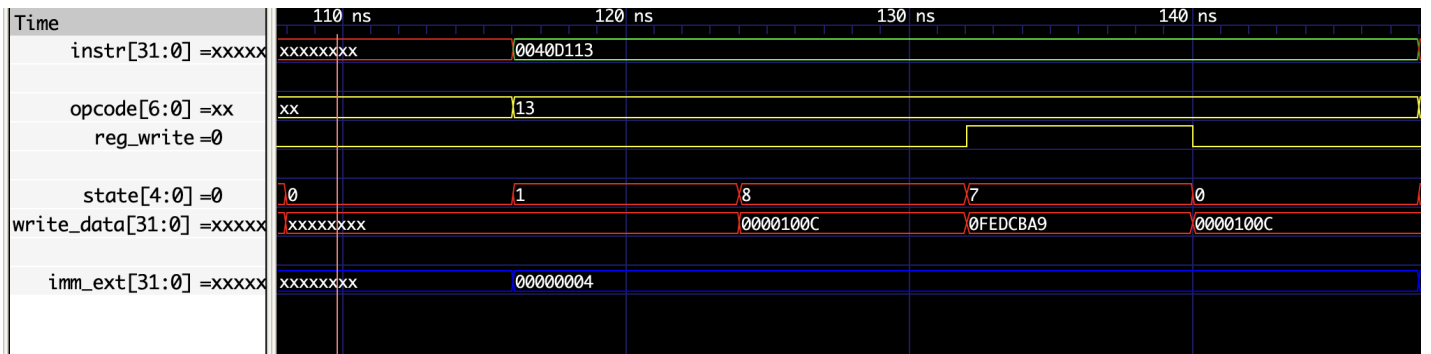


Figure 2. Our control unit manipulates the control signals highlighted in blue to configure the processor's datapath for each execution phase. The program starts in Fetch, where the PC address is routed to the Instruction Memory and IRWrite is asserted to retrieve the instruction. $PC + 4$ is concurrently calculated with ALUSrcA (PC) and ALUSrcB (4) and written back to PC via pc_update and pc_write. Subsequently, in Decode, different microstates are entered depending on the instruction field read from the IR. For a load/store instruction, a base register value from the Register File and immediate offset are added to compute a memory address. For R-/I- Type instructions, Execute occurs and ALU operations (depending on alu_control) are performed on ALUSrcA/B. Lastly, in Writeback, the RegWrite signal is asserted and the ResultSrc multiplexer selects the correct data to write into the Register File. Control flow instructions (ie. AUI, JAL, JALR, BEQ, and LUI) skip unnecessary states based on the operation required, optimizing the FSM's efficiency.

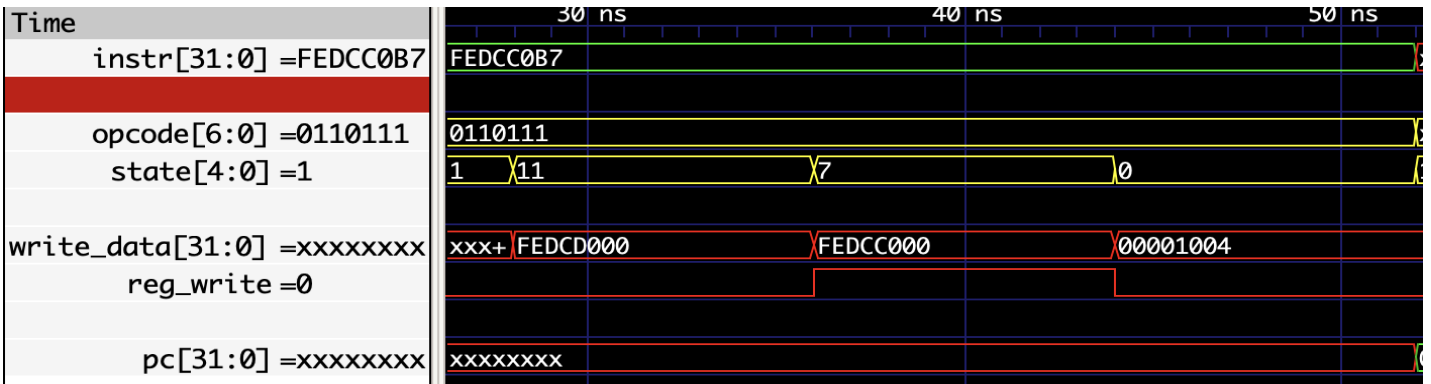
Timing Analysis

We generated simulations for each instruction type to observe how the processor completes the write-back phase. To debug behavior, we inspected the control unit and followed instruction flow through the datapath. The PC fetches an instruction, which is loaded into the IR via IRWrite. Operands or immediates then pass through the WriteData path and SrcB mux, using ImmExt when required. The ALU executes the operation, and its buffered/unbuffered outputs feed the final mux alongside memory data to produce Result written back to the register file. Below are corresponding simulations for sample instructions. The immediate address reflects our updated clock value and includes a 1 kB offset because the memory layout reserves the first 1024 bytes for command storage. In the test program, the jal instruction jumps forward by ten instructions, after which execution continues at a jalr with a program counter of 1038. Additionally, all instructions appear shifted by four clock cycles because the processor pauses at startup; we use those initial four cycles to assert the reset signal so that every module begins in a stable, defined state. Since all of the more complex R and I type instructions were verified, we are confident that the remaining instructions we don't show below will also work. See our simulations below:

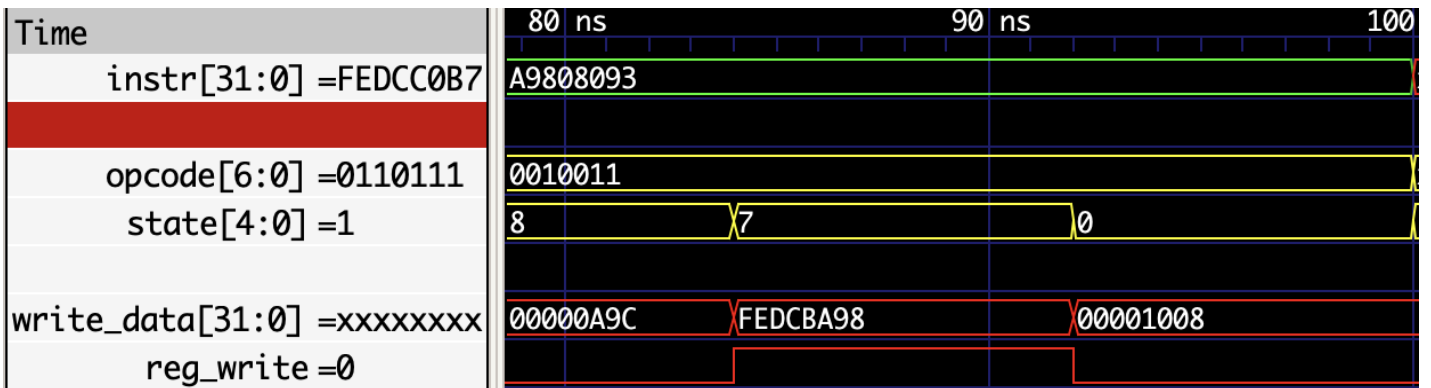
SRLI – S-type



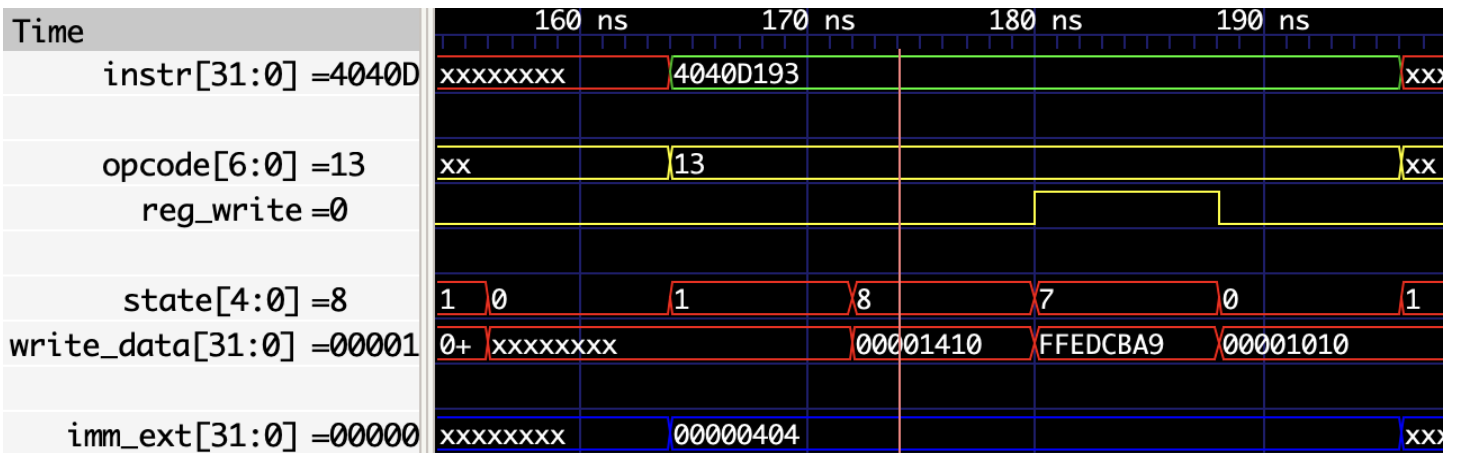
LUI – U-type



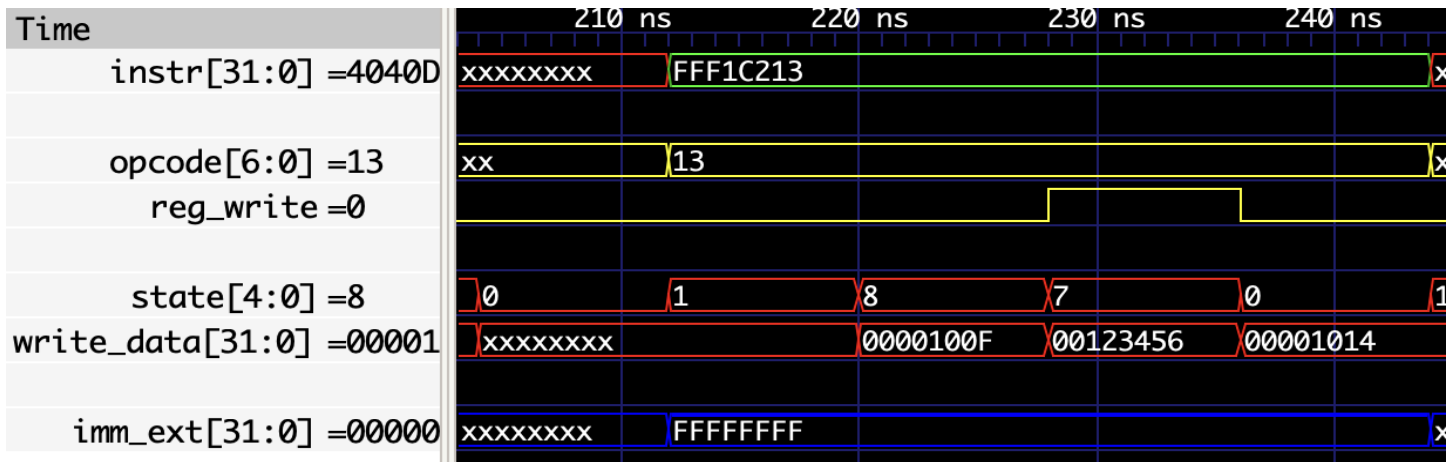
ADDI – R-type



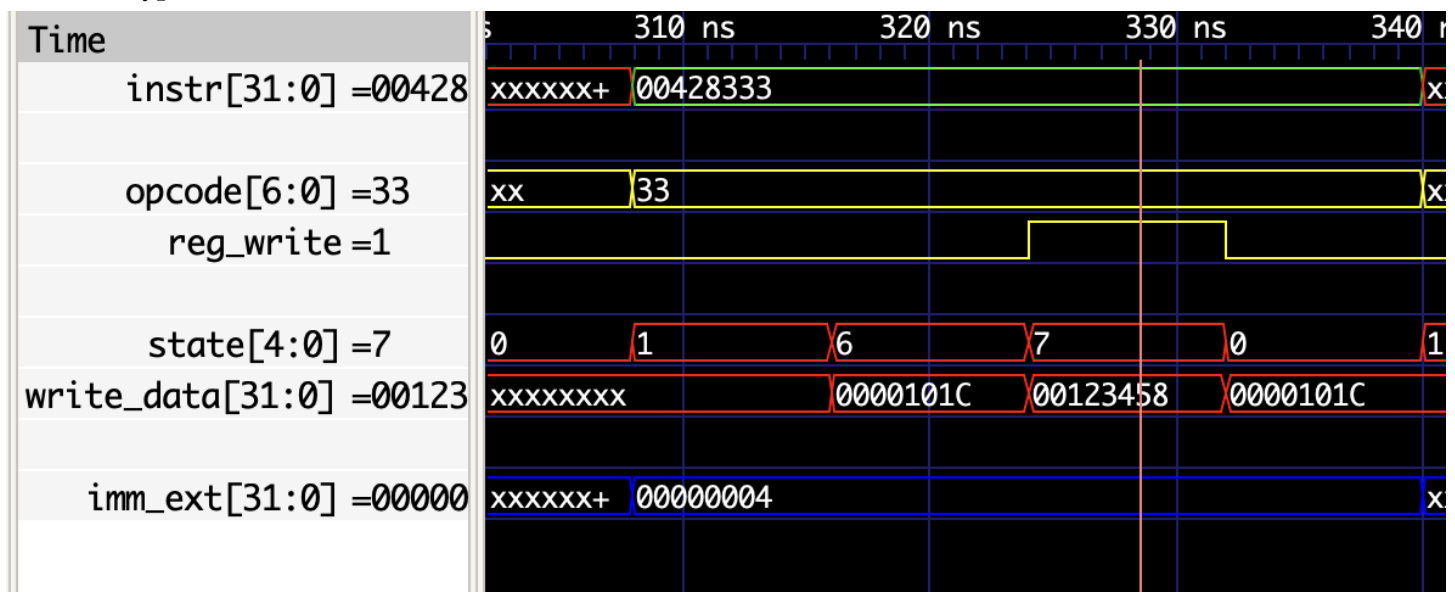
SRAI – S-type



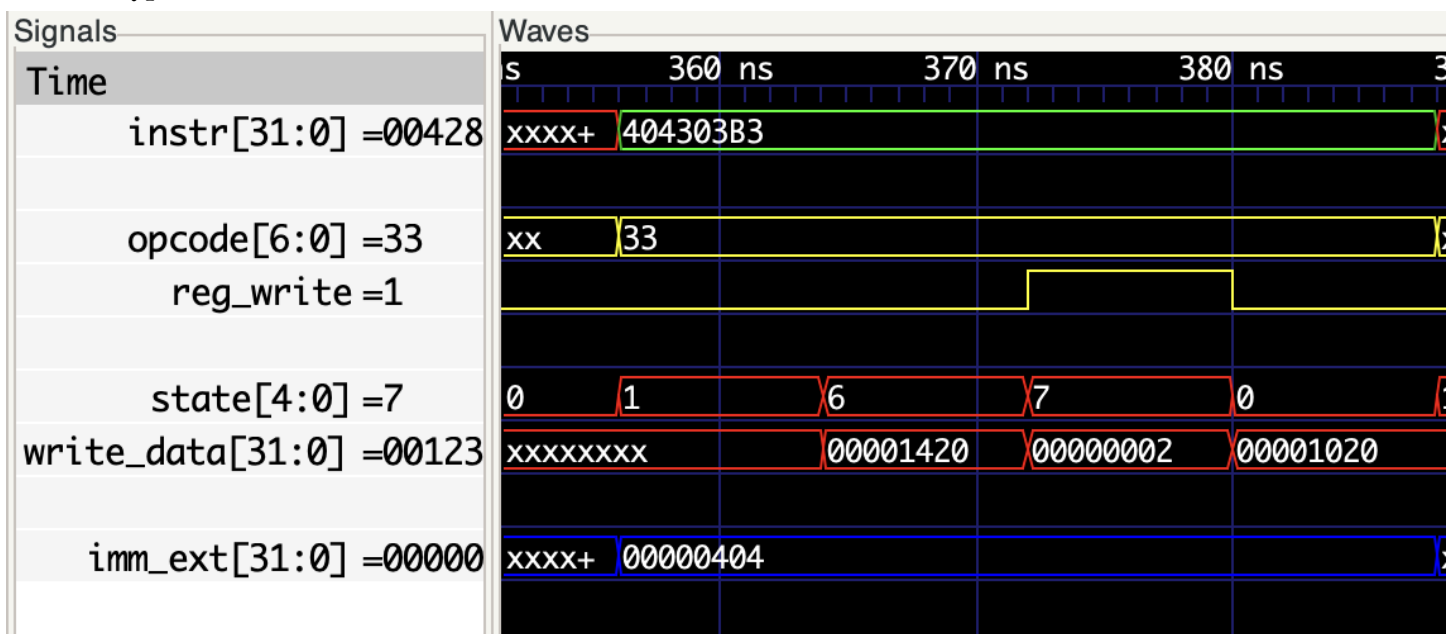
XORI – I-type



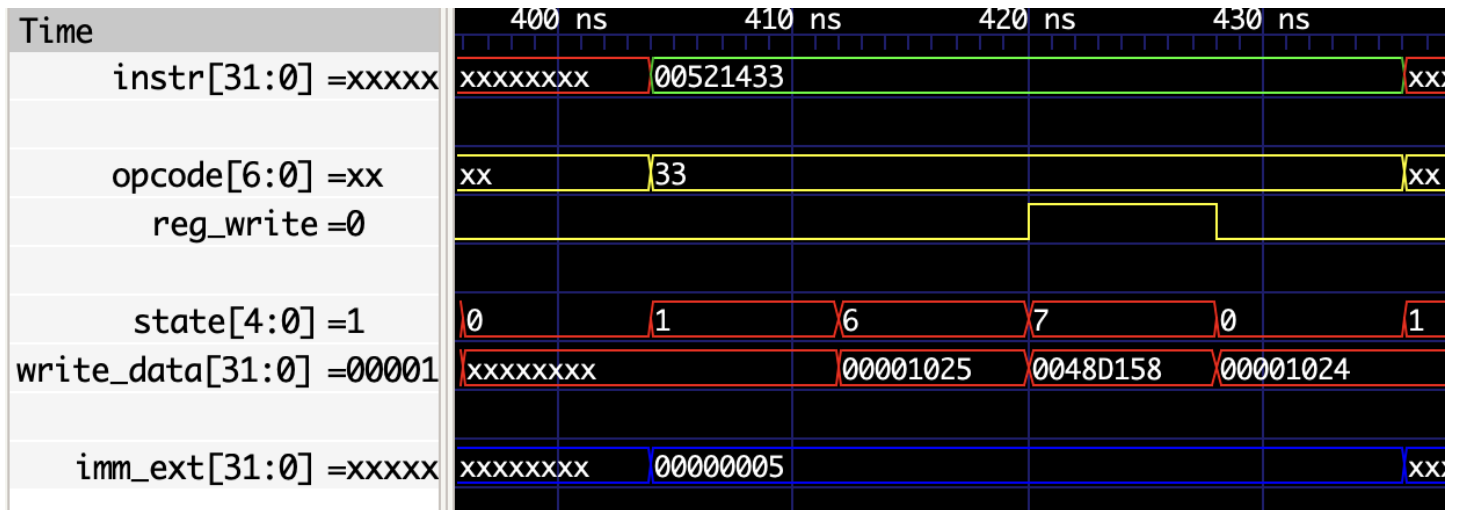
ADD – R-type



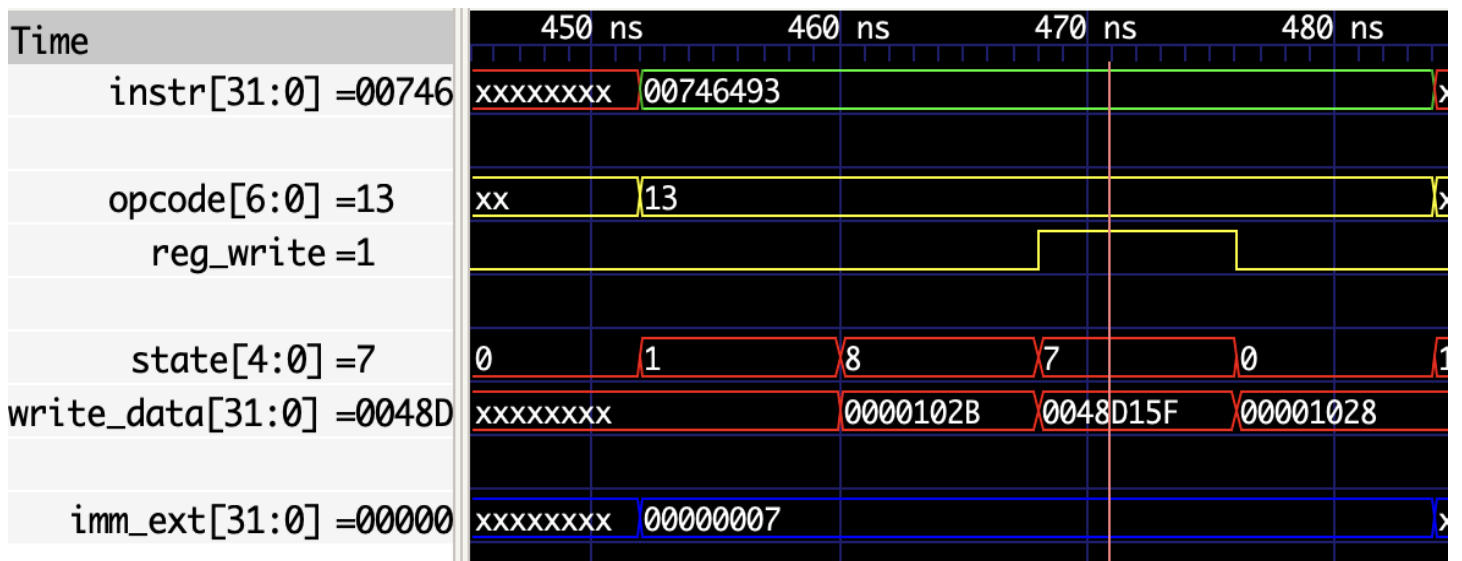
SUB – R-type



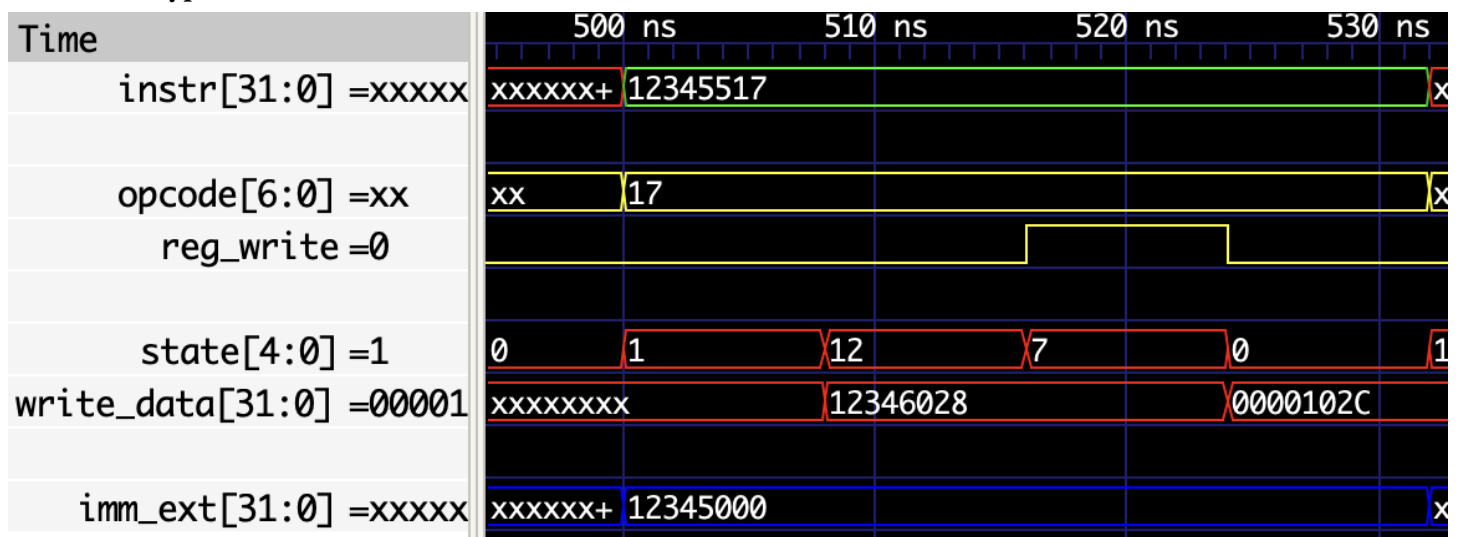
SLL – S-type



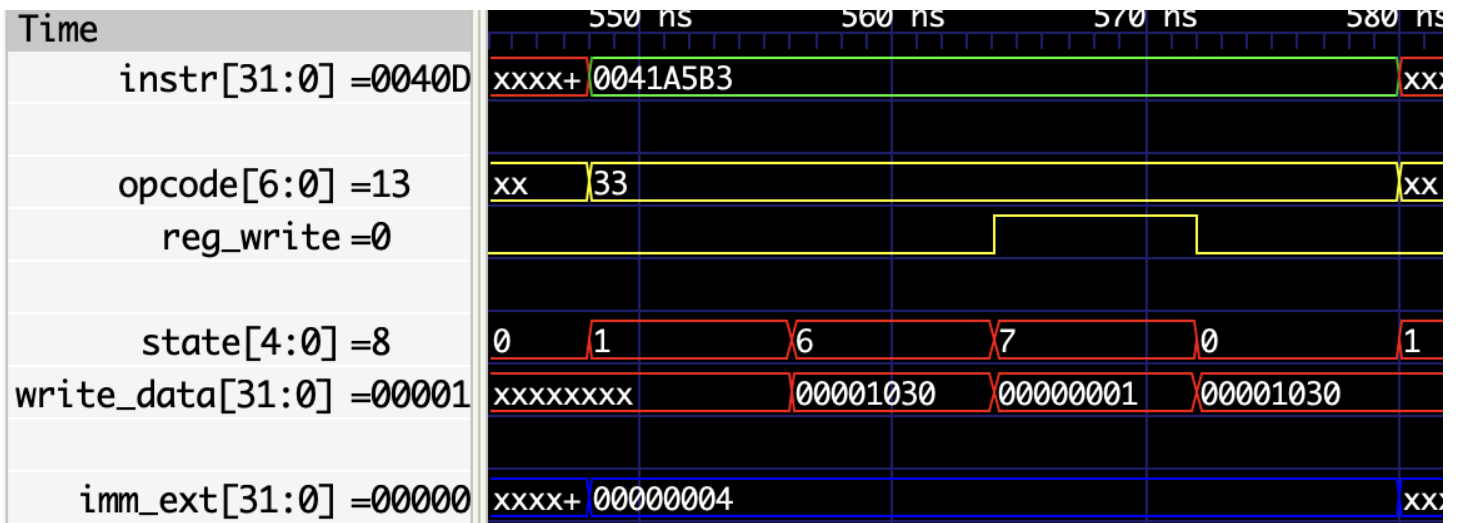
ORI – I-type



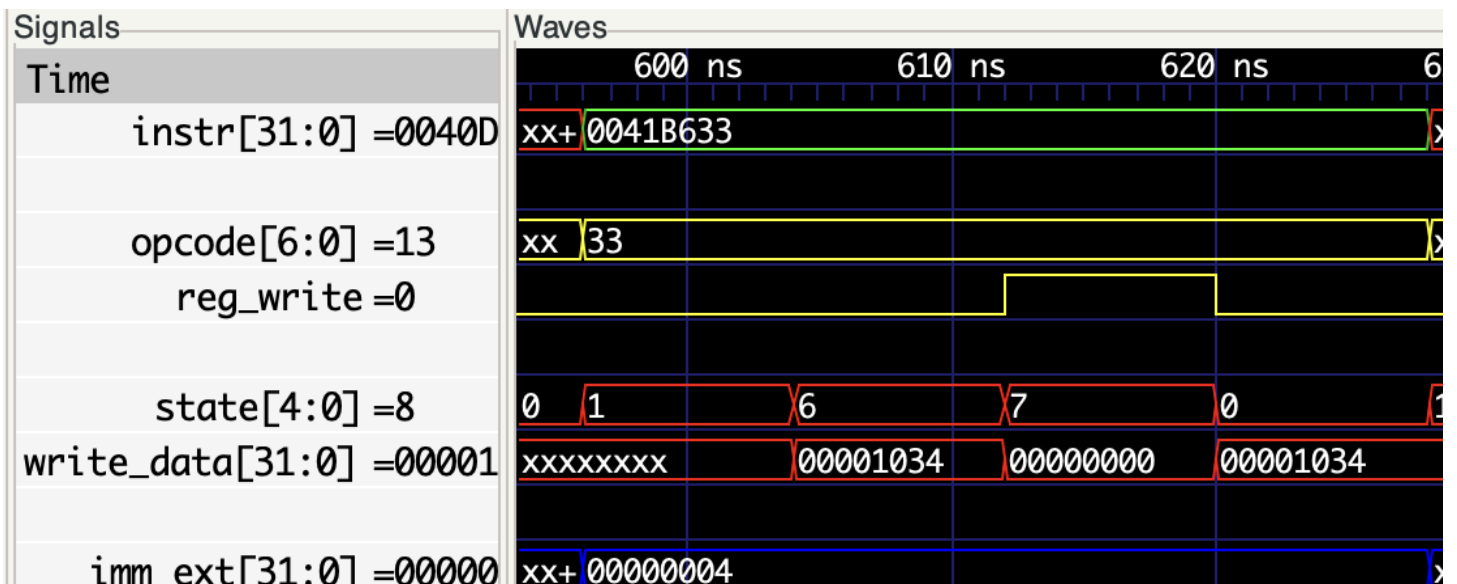
AUIPC – U-type



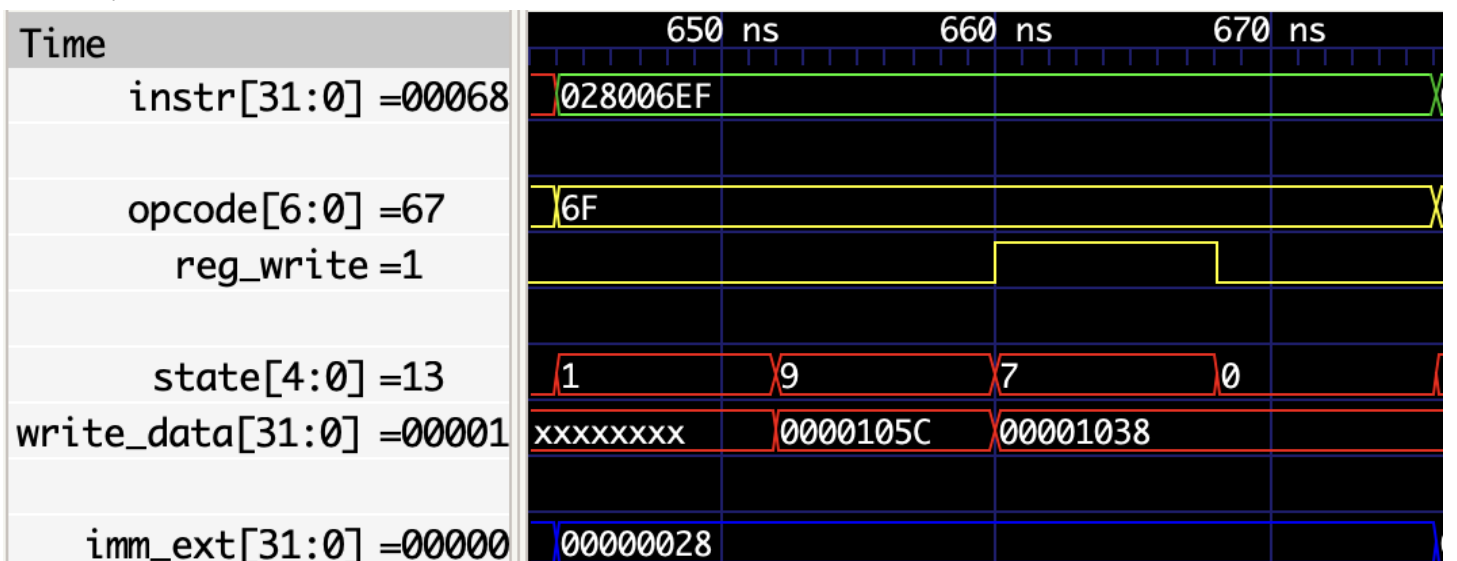
SLT – I-type



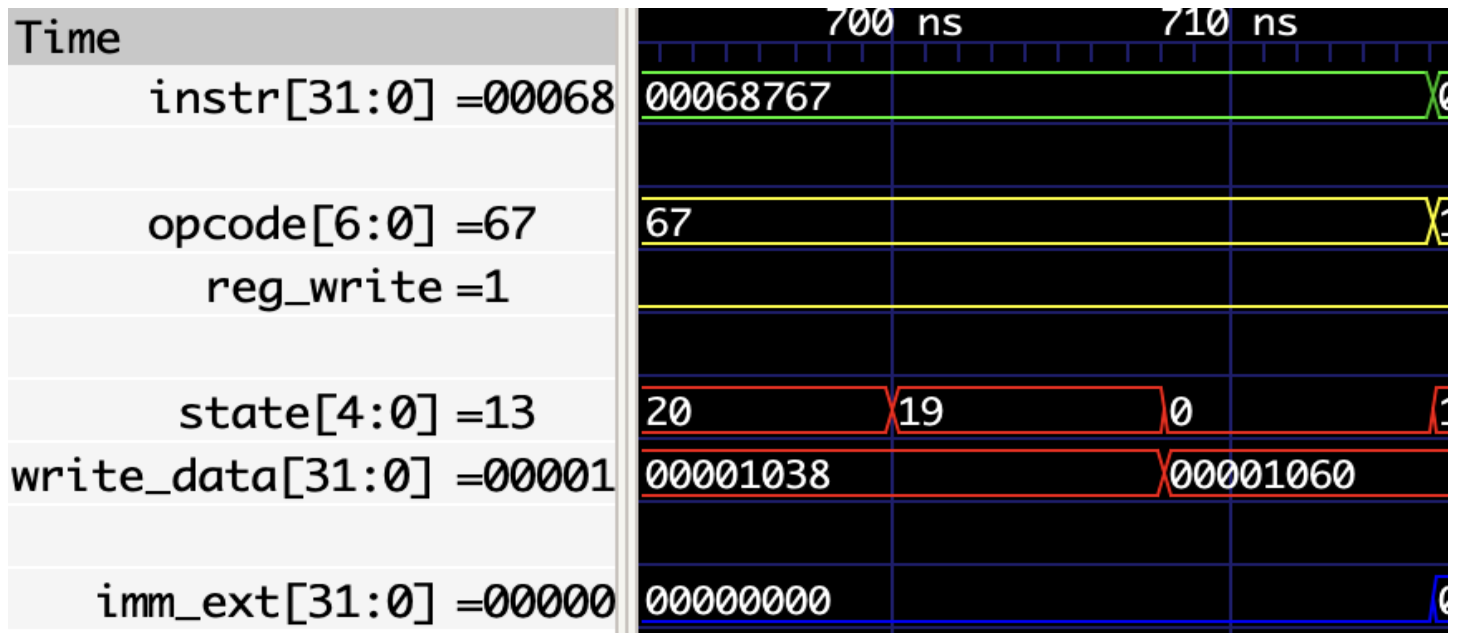
SLTU – I-type



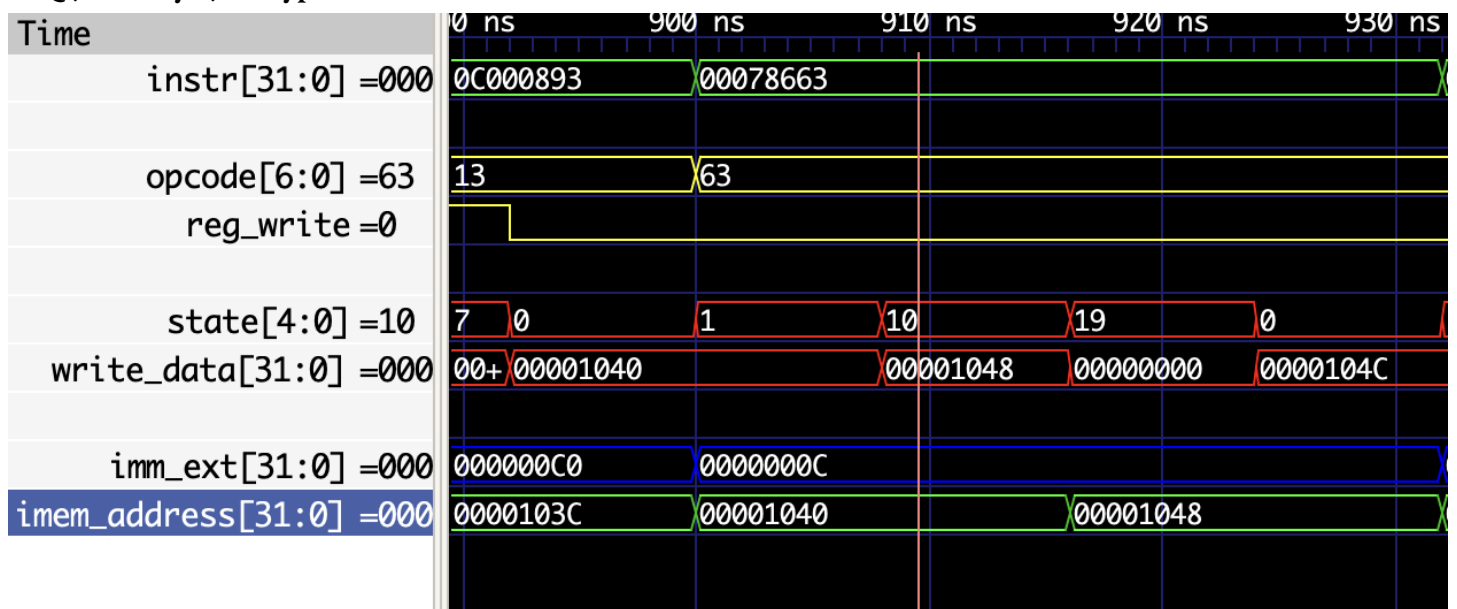
JAL – J-type



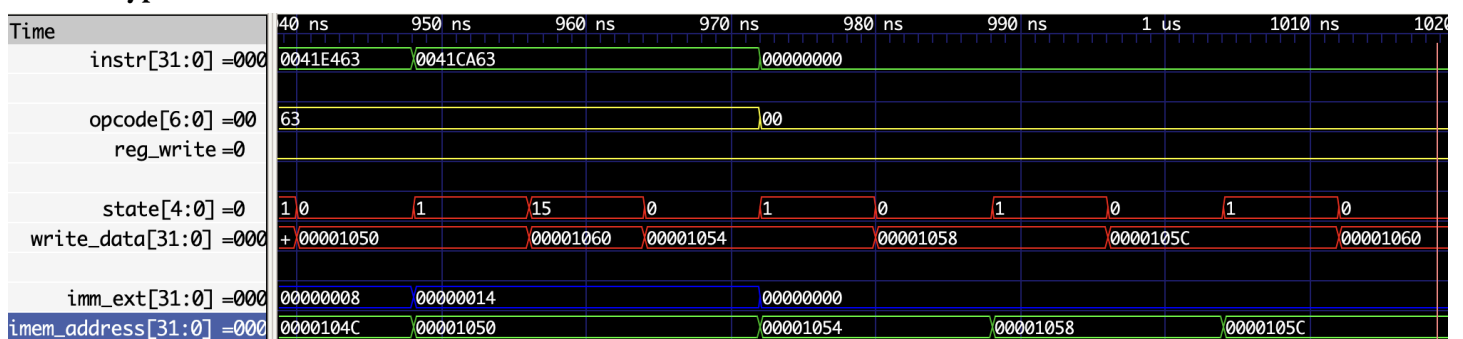
JALR – I-type



BEQ (Offset by 4) – I-type



BLT – B-type



BLTU – B-type

Time	40 ns
instr[31:0] =0041E463	0041E463
opcode[6:0] =63	63
reg_write =0	
state[4:0] =0	1 0
write_data[31:0] =00001050	+ 00001050
imm_ext[31:0] =00000008	00000008
imem_address[31:0] =0000104C	0000104C

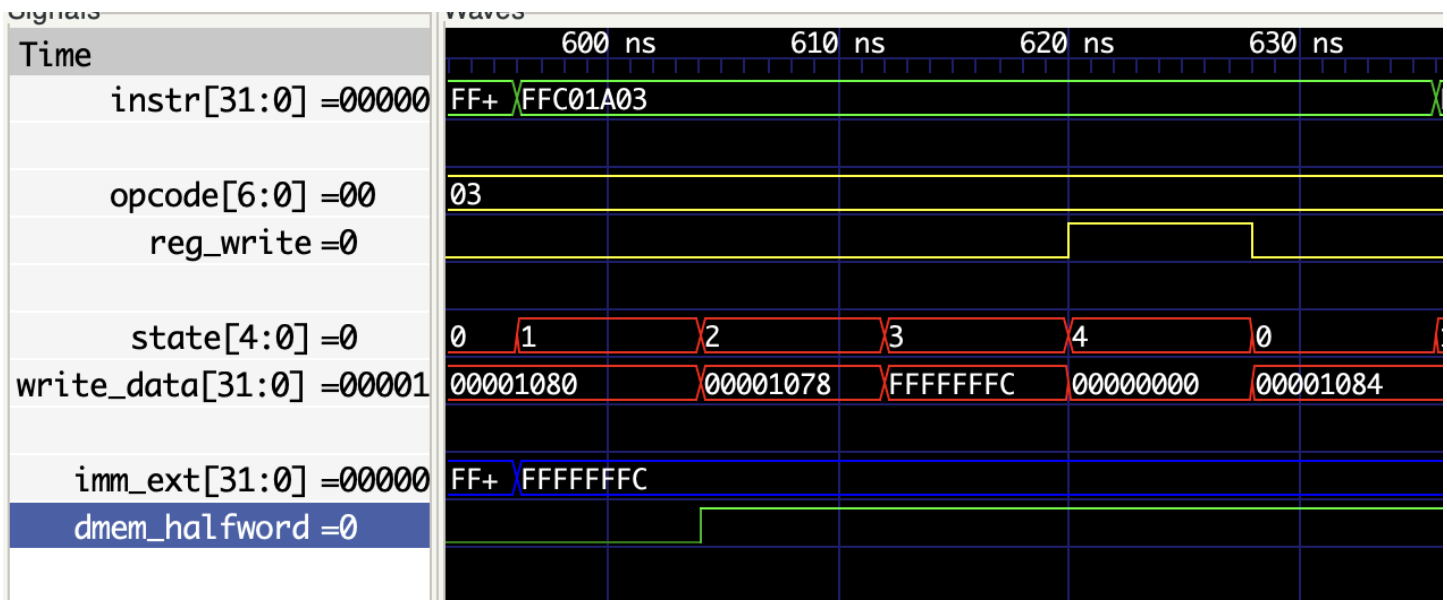
SB – S-type

Time	ns	430 ns	440 ns	450 ns
instr[31:0] =000000FF100E23	FF100E23			
opcode[6:0] =00	23			
reg_write =0				
state[4:0] =1	1	2	5	0
write_data[31:0] =00001068	00001068	00001060	FFFFFFFFC	0000106C
imm_ext[31:0] =000000FFFFFFFFC	FFFFFFFFC			
mem_write =0				

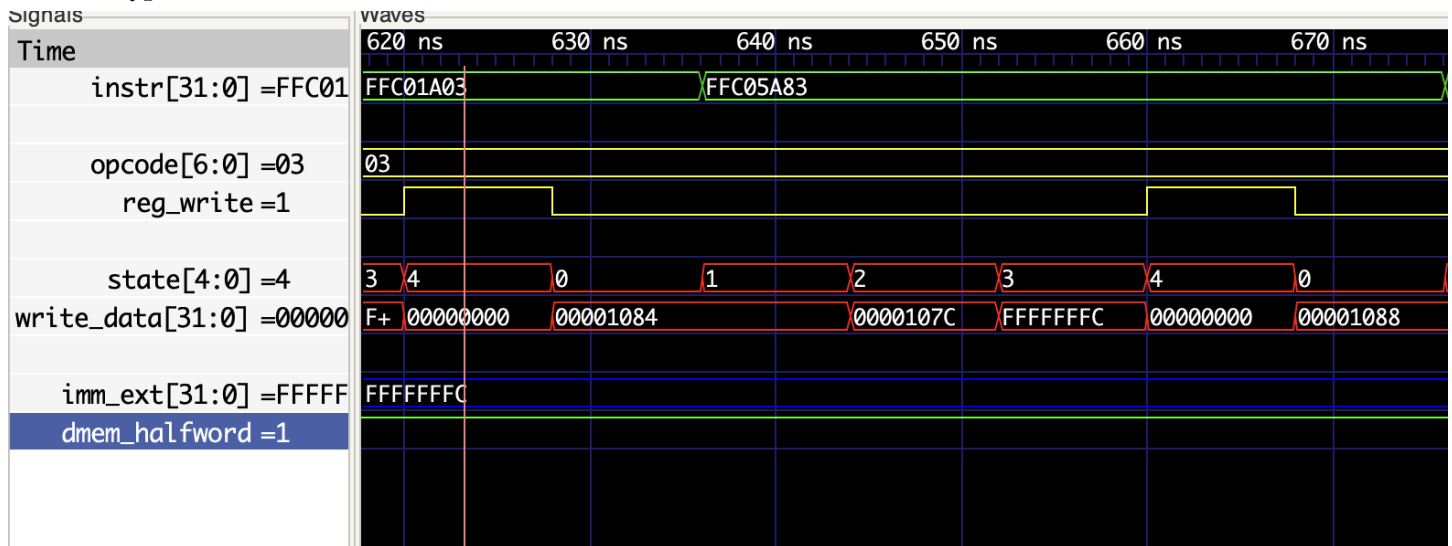
LW - I-type

Time	20 ns	530 ns	540 ns	550 ns
instr[31:0] =000000FFC02903	FFC02903			
opcode[6:0] =00	03			
reg_write =0				
state[4:0] =0	1	2	3	4
write_data[31:0] =00001070	00001070	FFFFFFFFC	00000000	0000107C
imm_ext[31:0] =000000FFFFFFFFC	FFFFFFFFC			
write_data[31:0] =00000000	00000000			

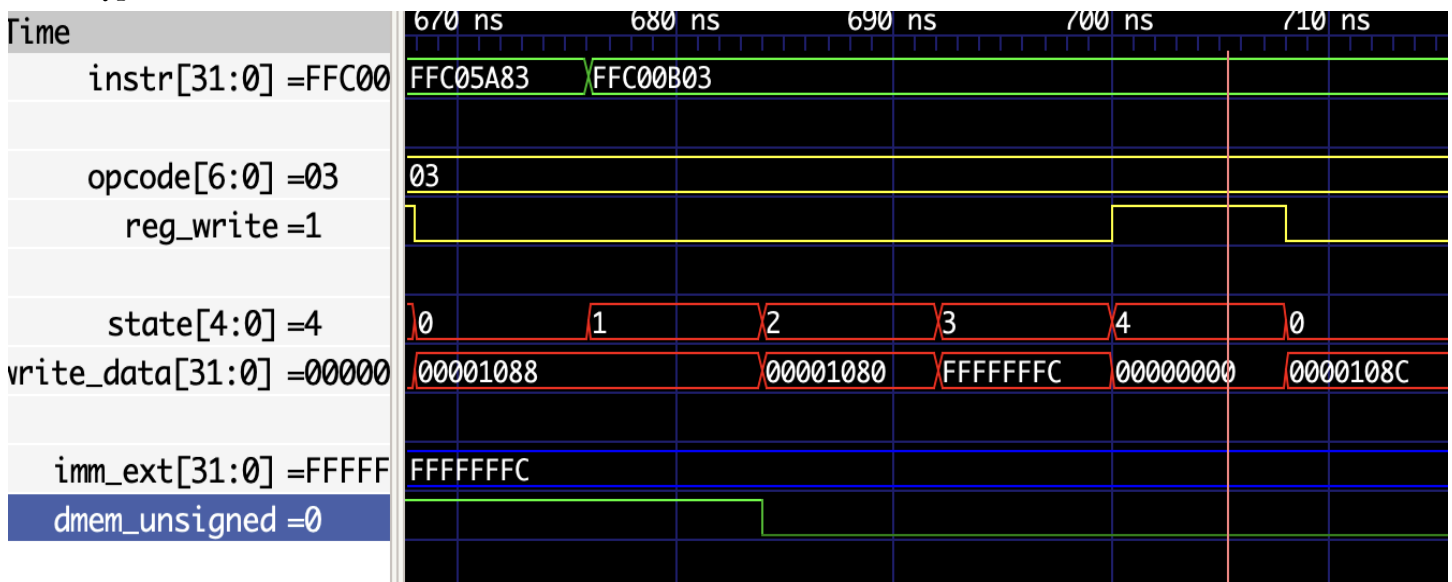
LH – I-type



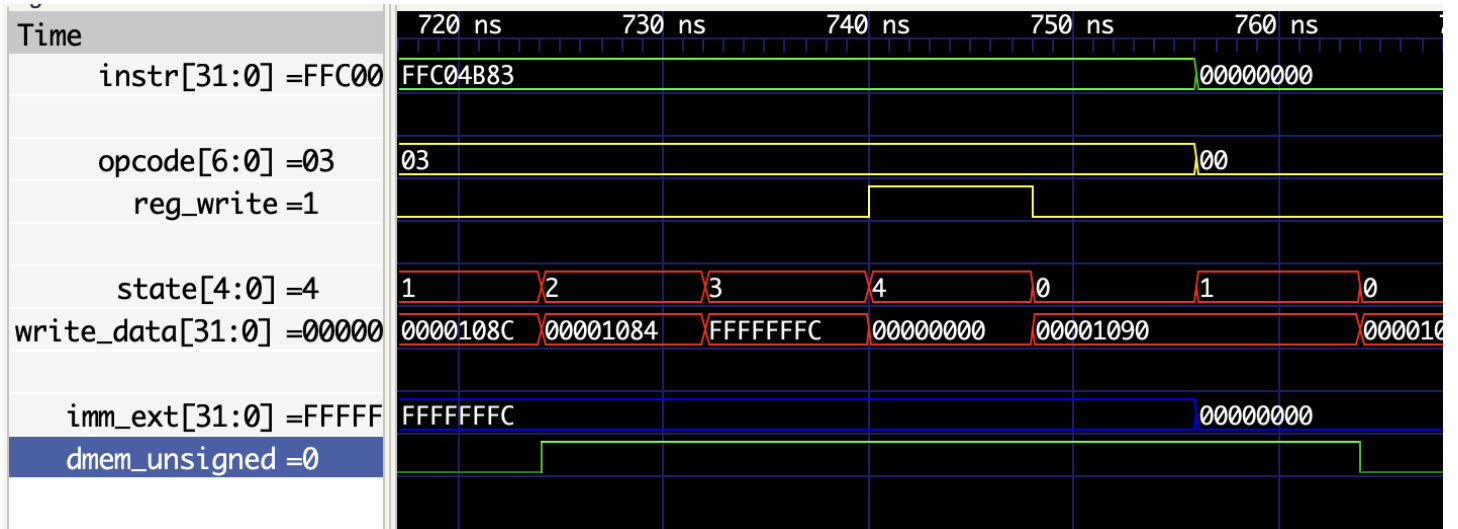
LHU – I-type



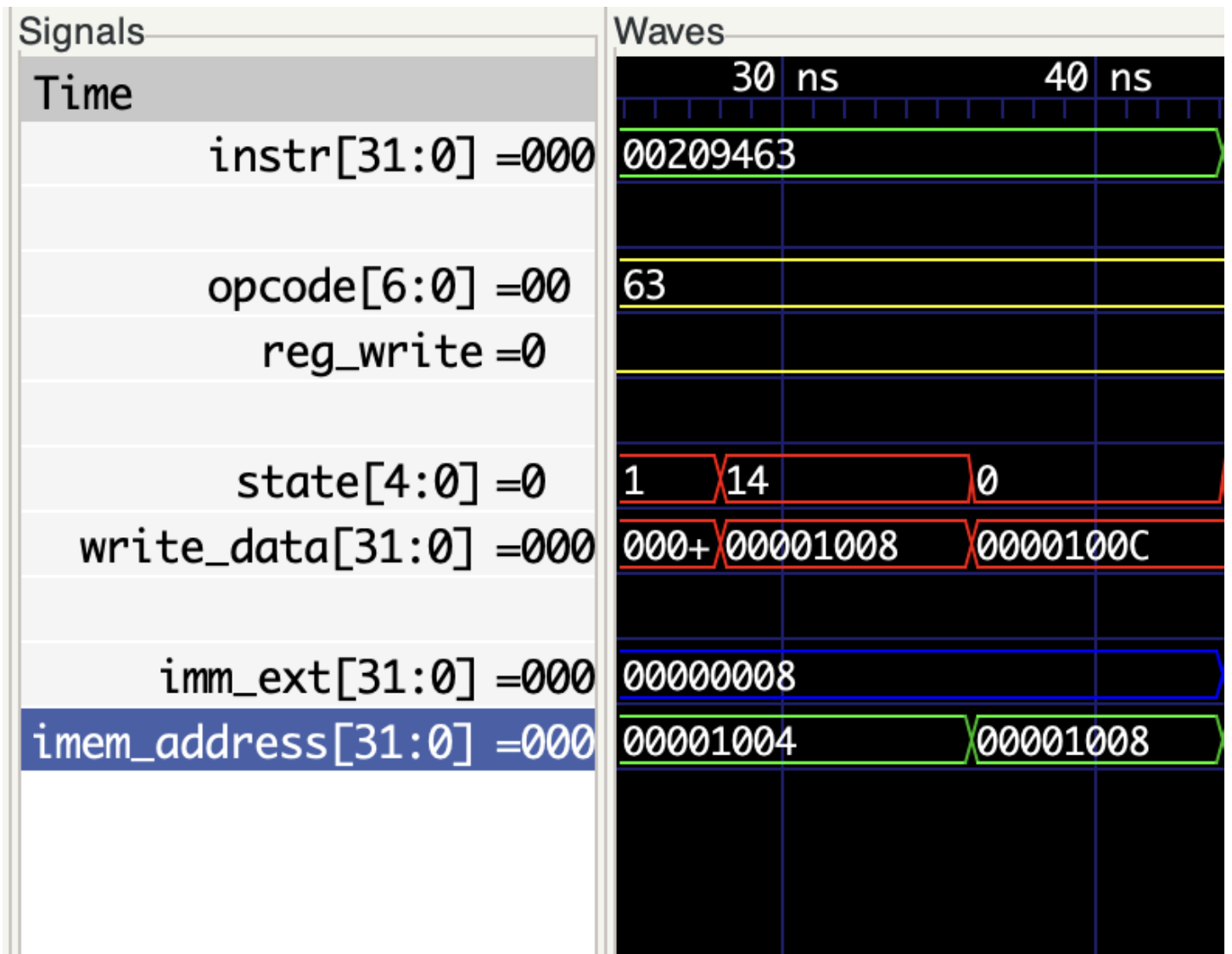
LB – I-type



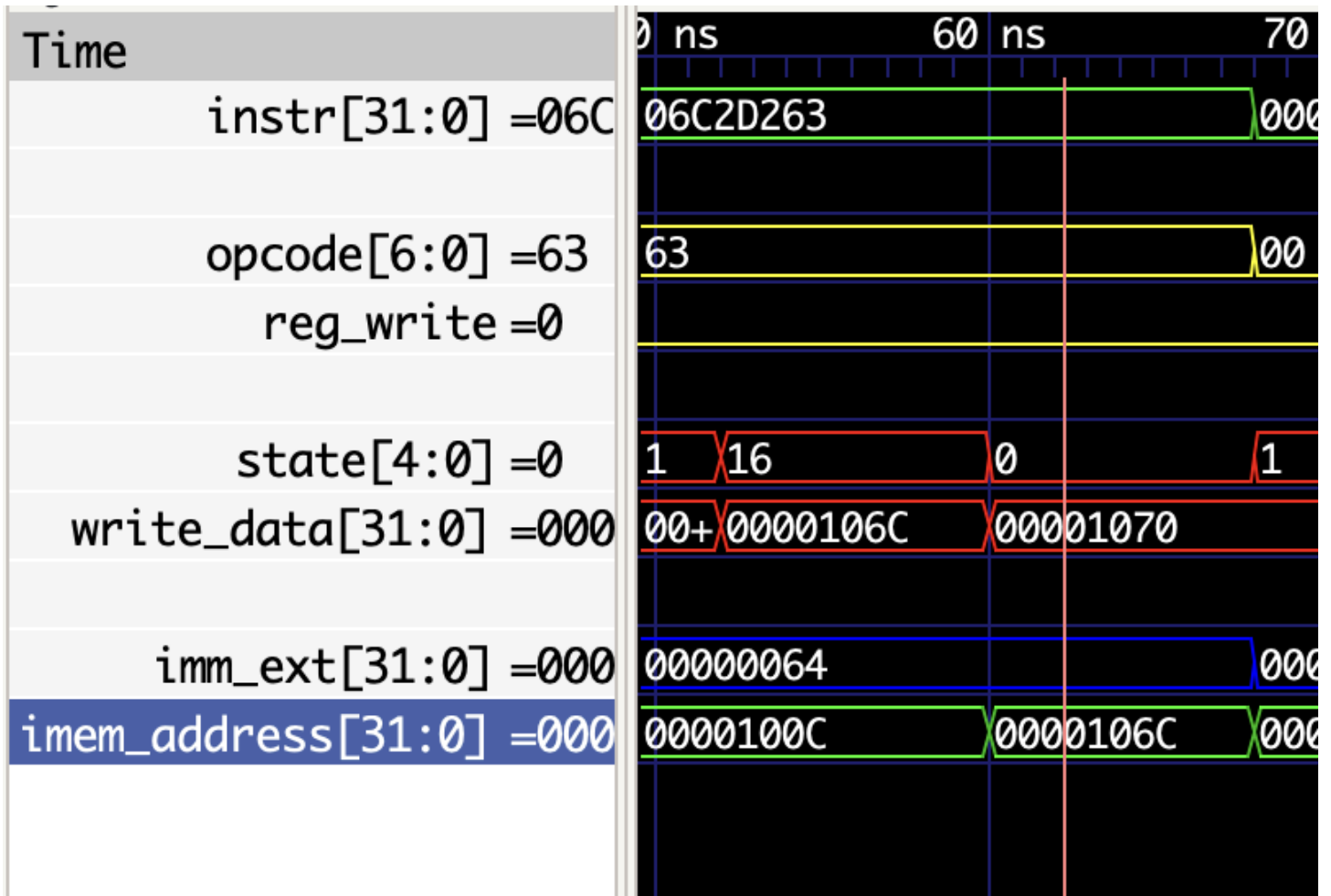
LBU – I-type



BNE – B-type



BGE – B-type



BGEU – B-type

